

Bandit - Level 11

Goal

The password for the next level is stored in `data.txt`, where all lowercase and uppercase letters have been rotated by 13 positions (ROT13).

Connection

```
ssh -p 2220 bandit11@bandit.labs.overthewire.org
```

Use the password retrieved at the end of [Bandit - Level 10](#).

Concepts

`tr` (translate) substitutes characters **positionally** between two sets: the 1st character of SET1 becomes the 1st of SET2, the 2nd becomes the 2nd, and so on. It operates character by character — it has no concept of words, patterns or lines.

```
tr SET1 SET2
```

Two properties of `tr` that cause most of the confusion around it:

- **Its arguments are the two character SETS, never the data.** The data always arrives via stdin.
- `tr` **accepts no filename argument at all.** Unlike `grep`, `sort`, `wc` or `head`, there is no way to hand it a file. `tr 'a' 'b' file.txt` is a syntax error, not a valid call.

That second point is the exact inverse of the trap in [Bandit - Level 09](#), where `sort data.txt` inside a pipeline *silently ignored* stdin because it had a file argument. `tr` is the opposite case: stdin is the only input it has.

Ranges are written with a hyphen, and multiple ranges concatenate:

```
'A-Za-z' → ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz (52 chars)
```

Attempts

```
cat data.txt
# → Gur cnffjbeq vf TEBbmJCB8DlA0zTewHxVQ0JPLxMvDkeA
# "Gur" is the tell - ROT13 of "The". Confirms the cipher before touching a tool.

cat data.txt | tr 'Gur cnffjbeq vf TEBbmJCB8DlA0zTewHxVQ0JPLxMvDkeA'
# → tr: missing operand after '...'
# Two strings must be given when translating.
# Passed the file's CONTENT as SET1. tr's arguments describe the transformation,
# not the data being transformed.

echo "data.txt" | tr 'Gur' 'cnffjbeq' 'vf' 'TEBbmJCB8DlA0zTewHxVQ0JPLxMvDkeA'
```

```
# → tr: extra operand 'vf'
# Two problems at once:
#   1. Four arguments – tr takes exactly two.
#   2. echo "data.txt" emits the literal seven-character string "data.txt",
#       not the file's contents. Quotes make it a string, not a file reference.
```

```
echo "data.txt" | tr 'TEBbmJCB8DlA0zTewHxVQ0JPLxMvDkeA'
# → tr: missing operand
# Still one set, still echoing the filename as text.
```

Solution

```
cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
# → The password is GROozWP08QyN0mGrjUkID0WCYkZiQxrN
```

Why that second set:

```
SET1:  A B C ... M  N O P ... Z   a b c ... m  n o p ... z
SET2:  N O P ... Z  A B C ... M   n o p ... z  a b c ... m
```

`N-Z` supplies the first 13 uppercase replacements (`A→N`, `B→O`, ... `M→Z`), then `A-M` supplies the next 13 (`N→A`, `O→B`, ... `Z→M`). `n-za-m` repeats it for lowercase. Both sets are 52 characters, so every letter has exactly one mapping.

Digits, spaces and punctuation appear in neither set, so `tr` passes them through untouched — which is why `8`, `0` and the spacing survive intact.

ROT13 is its own inverse. 13 is half of 26, so applying it twice returns the original. The same command encodes and decodes; there is no `-d` equivalent to forget, unlike [Bandit - Level 10](#).

Points of Friction

1. Passed the data as an argument instead of the transformation. The root misunderstanding: `tr`'s two arguments describe **how to transform**, not **what to transform**. Reading the error literally — *"Two strings must be given when translating"* — points at it: `tr` wanted two sets, and had been given one long string of file content.

2. `echo "data.txt"` prints the filename, not the file. Quoting a filename produces a literal string. Reading a file into a pipe requires `cat data.txt` or a redirect `< data.txt` — the distinction covered in [Linux - Piping and Redirection](#).

3. Three failed attempts before re-reading the error message. All three errors named the problem precisely (*missing operand*, *extra operand*, *two strings must be given*). The faster path was to stop after the first one and run `tr --help` — same lesson as the `-m32` detour in [Bandit - Level 10](#).

4. What went right: identifying the cipher before touching a tool. Spotting that `Gur` decodes to `The` confirmed ROT13 *before* any command was run. That's the same verify-before-acting method as `head -30` in [Bandit - Level 09](#) and `uniq -c` in [Bandit - Level 08](#) — the tooling wobbled, the diagnostic didn't.

Key Takeaway

`tr` translates characters by **position between two equal-length sets**, reads only from stdin, and accepts no file argument. Building the second set correctly *is* the puzzle — `'N-ZA-Mn-za-m'` is ROT13 expressed as a positional mapping rather than as arithmetic.

Conceptually this sits beside base64: ROT13 is an **obfuscation with no key**, reversible by anyone who recognises it. Two levels, two different mechanisms, one identical lesson — neither hides anything from anyone paying attention. Full treatment: [Linux - Encoding vs Encryption](#).

Next

```
ssh -p 2220 bandit12@bandit.labs.overthewire.org
```

◀ Previous: [Bandit - Level 10](#) · Next ▶ [Bandit - Level 12](#)